DUOC UC — ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES
CARRERA: INGENIERÍA EN INFORMÁTICA / ANALISTA PROGRAMADOR COMPUTACIONAL

INFORME TÉCNICO DE ASEGURAMIENTO DE CALIDAD — ESTÁNDAR IEEE 829

INFORME TÉCNICO DE COBERTURA Y TESTING UNITARIO

Evaluación Automatizada de Componentes React — Proyecto Pastelería Mil Sabores

Asignatura:	Desarrollo Fullstack II (DSY1104)
Evaluación:	Evaluación Parcial N° 2
Suite de Pruebas:	Karma 6.4.4 + Jasmine Core 5.1.2 + React Testing Library
Entorno de Ejecución:	Chrome Headless (Windows 10/11)
Resultado Consolidado:	**10 de 10 Pruebas Exitosas (100% de Aprobación — 0 Fallos)**
Tiempo de Ejecución:	0.117 segundos

1. Resumen Ejecutivo y Objetivos

El presente informe documenta el proceso de verificación técnica y pruebas unitarias automatizadas implementadas sobre el frontend en **React 18** de la tienda online **Pastelería Mil Sabores**. La suite de pruebas fue diseñada para satisfacer el 100% de los criterios de evaluación de la asignatura Desarrollo Fullstack II (DSY1104), abarcando:

1. **Renderizado de Listas:** Recorrido iterativo de colecciones y consistencia estructural en el DOM.
2. **Renderizado Condicional:** Visibilidad y supresión de elementos en función de estados lógicos.
3. **Transferencia de Propiedades (Props):** Paso de funciones callback y estructuras de datos complejas hacia componentes hijos.
4. **Gestión de Estado Interno (State):** Reactividad y mutación controlada con el hook `useState`.
5. **Simulación de Eventos de Usuario:** Ejecución desatendida de clics sintéticos y envíos de formulario con `fireEvent`.
6. **Aislamiento y Mocks (Spies):** Suplantación de capas de persistencia y comprobación matemática de lógica comercial.

2. Entorno y Configuración Técnica

La infraestructura de testing se integró en el archivo `karma.conf.cjs`, configurando Karma como ejecutor de pruebas, Jasmine como marco de aserciones y Webpack con Babel para la transpilación de JSX en Chrome Headless:

```
 // Configuración centralizada en karma.conf.cjs
module.exports = function (config) {
  config.set({
    frameworks: ['jasmine', 'webpack'],
    files: [
      { pattern: 'test/**/*.spec.jsx', watched: false }
    ],
    preprocessors: {
      'test/**/*.spec.jsx': ['webpack']
    },
    webpack: {
      mode: 'development',
      module: {
        rules: [
          {
            test: /.(js|jsx)$/,
            exclude: /node_modules/,
            use: {
              loader: 'babel-loader',
              options: {
                presets: [
                  ['@babel/preset-env', { targets: { chrome: '100' } }],
                  ['@babel/preset-react', { runtime: 'automatic' }]
                ]
              }
            }
          },
          {
            test: /.css$/,
            use: [path.resolve(__dirname, 'test/null-loader.cjs')]
          }
        ]
      }
    },
    browsers: ['ChromeHeadlessNoSandbox'],
    singleRun: true
  });
};
```

3. Matriz Consolidada de Pruebas Unitarias

N°	Criterio Evaluado	Archivo de Especificación	Componente / Módulo	Técnica Empleada	Resultado
01	Renderizado de Listas	`test/1_render_list.spec.jsx`	`ProductList`	Recorrido `.map()`	**Aprobado**
02	Renderizado Condicional	`test/2_render_conditional.spec.jsx`	`AlertMessage`	Condición booleana	**Aprobado**
03	Props en Componentes	`test/3_button_props.spec.jsx`	`Button`	Callback `onClick`	**Aprobado**
04	Props Complejas (Objetos)	`test/4_product_card_props.spec.jsx`	`ProductCard`	Desestructuración	**Aprobado**
05	Gestión de Estado (Inputs)	`test/5_form_state.spec.jsx`	`SearchForm`	Hook `useState`	**Aprobado**
06	Gestión de Estado (Contador)	`test/6_quantity_selector_state.spec.jsx`	`QuantitySelector`	Incremento/Decremento	**Aprobado**
07	Simulación de Eventos (Click)	`test/7_event_add_cart.spec.jsx`	`ProductCard`	`fireEvent.click`	**Aprobado**
08	Simulación de Eventos (Submit)	`test/8_event_form_submit.spec.jsx`	`SimpleContactForm`	`preventDefault`	**Aprobado**
09	Aislamiento con Spies / Mocks	`test/9_mock_crud_service.spec.jsx`	`productService`	`jasmine.spyOn`	**Aprobado**
10	Validación Lógica de Negocio	`test/10_mock_checkout_totals.spec.jsx`	`cartService`	Cálculos y Cupones	**Aprobado**

4. Detalle Técnico de los Casos de Prueba

Prueba 1: Renderizado Estructural de Colecciones

Objetivo: Validar que `ProductList` renderice exactamente el número de nodos correspondientes a la longitud del arreglo entregado.

```
it('renderiza la cantidad exacta de productos provistos en la lista', () => {
  const mockProducts = [
    { id: '1', nombre: 'Torta Amor', precio: 22000, categoria: 'Tortas Circulares' },
    { id: '2', nombre: 'Pie de Limón', precio: 3490, categoria: 'Postres Individuales' },
    { id: '3', nombre: 'Brazo de Reina', precio: 12900, categoria: 'Pastelería Tradicional' }
  ];
  render(<ProductList products={mockProducts} />);
  const items = screen.getAllByTestId('product-item');
  expect(items.length).toBe(3);
});
```

Prueba 2: Renderizado Condicional Basado en Propiedades

Objetivo: Confirmar que `AlertMessage` retorna `null` cuando la propiedad `message` está vacía y genera el contenedor accesible cuando recibe una cadena de texto.

```
it('no renderiza nada si no hay mensaje y se renderiza con el mensaje cuando existe', () => {
  const { container, rerender } = render(<AlertMessage message="" />);
  expect(container.firstChild).toBeNull();

  rerender(<AlertMessage message="Stock no disponible temporalmente" type="danger" />);
  const alert = screen.getByRole('alert');
  expect(alert).toBeTruthy();
  expect(alert.textContent).toContain('Stock no disponible');
});
```

Prueba 3 y 4: Propiedades Simples y Modelos Complejos

Objetivo: Asegurar la transmisión de eventos mediante callbacks en `Button` y el formateo de moneda nacional ($24.990) y badge de oferta en `ProductCard`.

```
it('ejecuta el callback onClick al presionar el botón', () => {
  const handleClick = jasmine.createSpy('handleClick');
  render(<Button label="Comprar Ahora" onClick={handleClick} variant="primary" />);
  fireEvent.click(screen.getByRole('button', { name: /Comprar Ahora/i }));
  expect(handleClick).toHaveBeenCalledTimes(1);
});
```

Prueba 5 y 6: Gestión del Estado Interno (State)

Objetivo: Comprobar la mutación de estado en componentes controlados (`SearchForm`) y verificar que el selector numérico (`QuantitySelector`) modifique el valor mostrado respetando cotas numéricas.

Prueba 7 y 8: Simulación de Eventos Sintéticos

Objetivo: Simular interacción del usuario mediante `fireEvent.click` al añadir un ítem al carrito y `fireEvent.submit` en el formulario de contacto verificando la neutralización de la recarga nativa de página.

Prueba 9 y 10: Aislamiento con Spies y Reglas Comerciales

Objetivo: Sustituir llamadas al sistema de almacenamiento con `spyOn(productService, 'getAll')` y verificar el algoritmo de tarificación: Subtotal ($31.970), descuento del 50% con cupón `FELICES50` ($15.985) y despacho gratis ($0 por superar $30.000).

```
it('calcula correctamente el subtotal, descuento del 50% y despacho gratis', () => {
  const items = [
    { id: 'TC001', precio: 24990, cantidad: 1 },
    { id: 'PI001', precio: 3490, cantidad: 2 }
  ];
  const totals = cartService.calculateTotals(items, 'FELICES50');
  expect(totals.subtotal).toBe(31970);
  expect(totals.descuento).toBe(15985);
  expect(totals.envio).toBe(0);
  expect(totals.total).toBe(15985);
});
```

5. Evidencia de Ejecución Oficial (Log de Terminal)

Registro emitido al ejecutar `npm test` en consola:

```
> pasteleria-mil-sabores-react@2.0.0 test
> karma start karma.conf.cjs --single-run

Webpack bundling...
asset commons.js 2.14 MiB [emitted] (name: commons) (id hint: commons)
asset runtime.js 9.16 KiB [emitted] (name: runtime)
webpack 5.112.0 compiled successfully in 2145 ms
INFO [karma-server]: Karma v6.4.4 server started at http://localhost:9876/
INFO [launcher]: Launching browsers ChromeHeadlessNoSandbox with concurrency unlimited
INFO [launcher]: Starting browser ChromeHeadless
INFO [Chrome Headless 156.0.0.0 (Windows 10)]: Connected on socket with id 21968276
Chrome Headless 156.0.0.0 (Windows 10): Executed 1 of 10 SUCCESS (0.025 secs)
Chrome Headless 156.0.0.0 (Windows 10): Executed 2 of 10 SUCCESS (0.030 secs)
Chrome Headless 156.0.0.0 (Windows 10): Executed 3 of 10 SUCCESS (0.057 secs)
Chrome Headless 156.0.0.0 (Windows 10): Executed 4 of 10 SUCCESS (0.072 secs)
Chrome Headless 156.0.0.0 (Windows 10): Executed 5 of 10 SUCCESS (0.074 secs)
Chrome Headless 156.0.0.0 (Windows 10): Executed 6 of 10 SUCCESS (0.079 secs)
Chrome Headless 156.0.0.0 (Windows 10): Executed 7 of 10 SUCCESS (0.086 secs)
Chrome Headless 156.0.0.0 (Windows 10): Executed 8 of 10 SUCCESS (0.087 secs)
Chrome Headless 156.0.0.0 (Windows 10): Executed 9 of 10 SUCCESS (0.089 secs)
Chrome Headless 156.0.0.0 (Windows 10): Executed 10 of 10 SUCCESS (0.090 secs)
TOTAL: 10 SUCCESS (0.117 secs / 0.090 secs)
```

6. Mapeo con Indicadores de Logro (Rúbrica Duoc UC)

Indicador	Criterio Evaluado	Evidencia Técnica en el Proyecto	Cumplimiento
IE2.2.1	Implementa pruebas unitarias para componentes básicos de interfaz gráfica.	Tests 1 al 4 validan renderizado de listas, condicionales y paso de propiedades.	**100% Logrado**
IE2.2.2	Evalúa la interacción del usuario y cambios de estado en componentes React.	Tests 5 al 8 simulan mutaciones de `useState` y eventos sintéticos de formulario.	**100% Logrado**
IE2.3.1	Aplica mocks y stubs para aislar dependencias externas y servicios de datos.	Test 9 implementa `jasmine.spyOn` aislando la persistencia de `productService`.	**100% Logrado**
IE2.3.2	Verifica lógica de negocio compleja, cálculos matemáticos y validaciones.	Test 10 certifica fórmulas comerciales, aplicación de cupones y umbrales de despacho.	**100% Logrado**

Responsable de Aseguramiento de Calidad (QA)
Equipo de Desarrollo Fullstack — Duoc UC

Docente Evaluador DSY1104
Escuela de Informática y Telecomunicaciones